

An In-Network Timing Framework for Concealing the Response-Time Fingerprint of a Separate-Acknowledgment DNP3 Device

Anonymous submission

I. Introduction

An attacker studies a control system before it acts on it. Fingerprinting is how a passive observer turns ordinary network traffic into knowledge about the devices that produce it, learning a device's type, vendor, and role without decoding any payload [1]–[3]. In industrial control systems the payoff is large and durable, because the field devices are long lived and their roles are fixed, and prior work has fingerprinted supervisory and field equipment directly from its traffic [4], [5]. One of the features that gives a device away is timing. A DNP3 outstation answers a master's poll with a transport acknowledgment and then, a short and characteristic delay later, the response itself. That delay, the cross-layer response time, distinguishes control-system devices by vendor and model [5]. Because DNP3 carries it in the clear, an on-path observer reads the fingerprint from routine polling traffic.

The obvious answer is traffic obfuscation, and there is a large toolkit for it: add cover packets, pad sizes, and reshape flows so the observer's measurements no longer separate one source from another [6]–[9]. That toolkit does not transfer to industrial control, and it is worth stating plainly why, because the reason shapes the rest of the paper. General obfuscation was built for the encrypted Internet, where the traffic is opaque and the shaper is free to invent packets and reshape flows because the observer cannot tell a real exchange from a manufactured one [6], [8], and where the weaker of these countermeasures have in any case been broken by stronger analysis [10], [11]. Legacy industrial traffic inverts every assumption those methods rest on:

- It is plaintext and checksummed. A DNP3 frame is readable and each link block carries a validating CRC [12], [13], so an injected dummy or an altered byte is detectable by inspection, not hidden by encryption. Cover traffic and byte edits, the staples of general obfuscation, are off the table.
- The endpoints cannot be changed. The outstation is vendor firmware and DNP3 is a deployed standard [12], [14], so a defense cannot ask the device or the protocol to shape their own traffic.

- Correctness is not optional. Every poll must receive its real response, in order, within the protocol's timing, retransmission, and quality-of-service bounds. A defense cannot drop, reorder, or delay past those bounds without breaking the control loop.
- The fingerprint is in the timing. Size- and volume-oriented defenses do not touch the response-time signature that identifies the device.

The consequence is a specific and different problem. A defense for this setting must live in the network, because the device and the protocol are fixed; it must preserve every original byte, because the traffic is plaintext and checksummed; it must preserve the exchange and the correctness of every delivered reply; and it must act on the timing observable itself, which is the thing that leaks. None of the general obfuscation methods meets all four constraints at once, and that gap, not a missing tuning of an existing method, is what this paper addresses.

We present an in-network defense that meets the four constraints. One programmable switch sits between the DNP3 master and the field device and changes only when the device's acknowledgment and response become externally visible, never their contents. Rather than editing any packet, it holds the original acknowledgment and response in its queues while small internal marker packets recirculate to keep their place, and the traffic manager releases the held packets on a schedule the control plane sets [15]–[17]. This design works with, rather than against, the fact that a data-plane pipeline cannot pause an enqueued packet and recall it later at a software-chosen time. Configurable gates on the acknowledgment and on the response give a small family of timing transformations, from releasing the acknowledgment only when its matching response is ready, to holding the acknowledgment or the response to a fixed deadline, to holding both, all as modes of one mechanism selected at run time. We assume the switch is a trusted observation and control point on the plaintext path, and that the adversary is a passive observer on the master-facing segment, where only the released, shaped timing is visible.

Contributions. Each contribution is bounded by the evidence we report.

- We state why general traffic obfuscation does not work for legacy ICS traffic, and we frame the cross-layer response time, identified by Formby et al. [5] as a device-fingerprinting feature, as a reachable in-network shaping target under four constraints, in-network, byte-preserving, correctness-preserving, and timing-targeted, that separate this problem from the size- and encryption-oriented defenses above.
- We design a unified, queue-resident timing framework whose event, acknowledgment-deadline, response-deadline, and dual-deadline transformations are configurations of one mechanism. A separate-acknowledgment device releases its acknowledgment before its response, so the response obligation must survive the acknowledgment release; we show how the transaction state is kept alive across that release so a later response is still held rather than forwarded unshaped.
- We resolve the DNP3, TCP, and Tofino constraints the design meets: matching each transaction to its DNP3 transaction-generation code on plaintext traffic, keeping the internal markers off the observable wire, preserving the original bytes, and fitting the arithmetic within a single Tofino-1 ingress pipeline.
- We implement the framework on an Intel Tofino-1 switch and evaluate it on silicon against a physical SEL-751 relay. On the corrected implementation, the response-deadline and dual-deadline modes normalize the measured response time to a fixed value and hold every response, verified by a fail-closed measurement pipeline and reproduced across two campaigns; we quantify the resulting collapse of the timing observable. We report these results, and their limits, for one relay, for READ polling transactions, and for the response-time observable only. We do not claim device indistinguishability across vendors, size concealment, or the full set of controlled failure cases; the controlled negative-path tests and a cross-device classification study are stated as future work, not as results.

References

- [1] T. Kohno, A. Broido, and K. C. Claffy, "Remote physical device fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [13].
- [2] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A technique for physical device and device type fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, 2014, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [18].
- [3] G. Shu and D. Lee, "Network protocol system fingerprinting – a formal approach," in *Proceedings IEEE INFOCOM 2006*, 2006, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [19].
- [4] S. Jeon, J.-H. Yun, S. Choi, and W.-N. Kim, "Passive fingerprinting of SCADA in critical infrastructure network without deep packet inspection," 2016, verified via arXiv MCP (export_citations). [Online]. Available: <https://arxiv.org/abs/1608.07679>
- [5] D. Formby, P. Srinivasan, A. Leonard, J. Rogers, and R. Beyah, "Who's in control of your control system? device fingerprinting for cyber-physical systems," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2016, anchor paper, read in full. The CLRT device-fingerprinting threat model.
- [6] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009, metadata verified from Ditto (NDSS 2022) bibliography, ref. [100].
- [7] X. Cai, R. Nithyanand, and R. Johnson, "CS-BuFLO: A congestion sensitive website fingerprinting defense," in *Proceedings of the 13th Workshop on Privacy in the Electronic Society (WPES)*, 2014, venue corroborated by Ditto (NDSS 2022) ref. [36]; metadata via arXiv MCP (1401.6022, "New Approaches to Website Fingerprinting Defenses").
- [8] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Computer Security – ESORICS 2016*, 2016, WTF-PAD. Venue corroborated by Ditto (NDSS 2022) ref. [67]; metadata via arXiv MCP (1512.00524).
- [9] N. Aphorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the smart home private with smart(er) IoT traffic shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2019, stochastic Traffic Padding (STP). Venue corroborated by Ditto (NDSS 2022) ref. [26]; metadata via arXiv MCP (1812.00955).
- [10] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail," in *2012 IEEE Symposium on Security and Privacy (S&P)*, 2012, buFLO. Metadata verified from Ditto (NDSS 2022) bibliography, ref. [50].
- [11] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, venue corroborated by Ditto (NDSS 2022) ref. [92]; metadata via arXiv MCP (1801.02265).
- [12] S. East, J. Butts, M. Papa, and S. Sheno, "A taxonomy of attacks on the DNP3 protocol," in *Critical Infrastructure Protection III (IFIP AICT, vol. 311)*. Springer, 2009, pp. 67–81, web-verified (Springer / IFIP CIP III).
- [13] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 state-based intrusion detection system," in *2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [10].
- [14] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks against process control systems: Risk assessment, detection, and response," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [6].
- [15] L. Wang, H. Kim, P. Mittal, and J. Rexford, "Programmable in-network obfuscation of traffic," 2020, verified via arXiv MCP (get_abstract). PINOT; Tofino prototype. [Online]. Available: <https://arxiv.org/abs/2006.00097>
- [16] R. Meier, V. Lenders, and L. Vanbever, "ditto: WAN traffic obfuscation at line rate," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022, anchor paper, read in full. Closest in-network line-rate obfuscation prior work.
- [17] E. F. Kfoury, J. Crichigno, and E. Bou-Harb, "An exhaustive survey on P4 programmable data plane switches: Taxonomy, applications, challenges, and future trends," 2021, verified via arXiv MCP (export_citations). Also in *IEEE Access*, 2021. [Online]. Available: <https://arxiv.org/abs/2102.00643>